

Distributed data store

- ▶ Basic problem statement: given a set of storage nodes, we need to store a set of objects on them, and retrieve them upon request
- ▶ Idea: have a consistent hashing function $h(x)$. It maps an object to a number i , and stores it on node i .
- ▶ Design objectives:
 - ▶ Deterministic, Efficient, Load balancing (Why?)

Chord

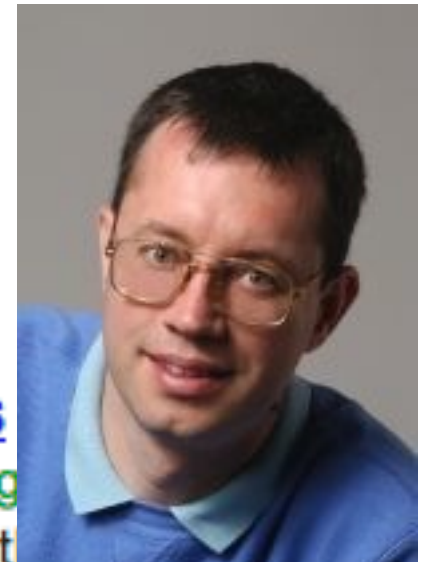
- ▶ A distributed hash table widely used in distributed systems
- ▶ Invented by Ion Stoica, Robert Morris, Hari Balakrishnan et al. in 2001 (Sigcomm 2001)
- ▶ One of the most influential work in CS

[Chord: A scalable peer-to-peer lookup service for internet applications](#)

I Stoica, R Morris, [D Karger](#), MF Kaashoek... - ACM SIGCOMM ..., 2001 - dl.acm.org

Abstract A fundamental problem that confronts peer-to-peer applications is to efficiently locate the node that stores a particular data item. This paper presents Chord, a distributed lookup protocol that addresses this problem. Chord provides support for just one ...

[Cited by 11202](#) [Related articles](#) [Cite](#) [Save](#)



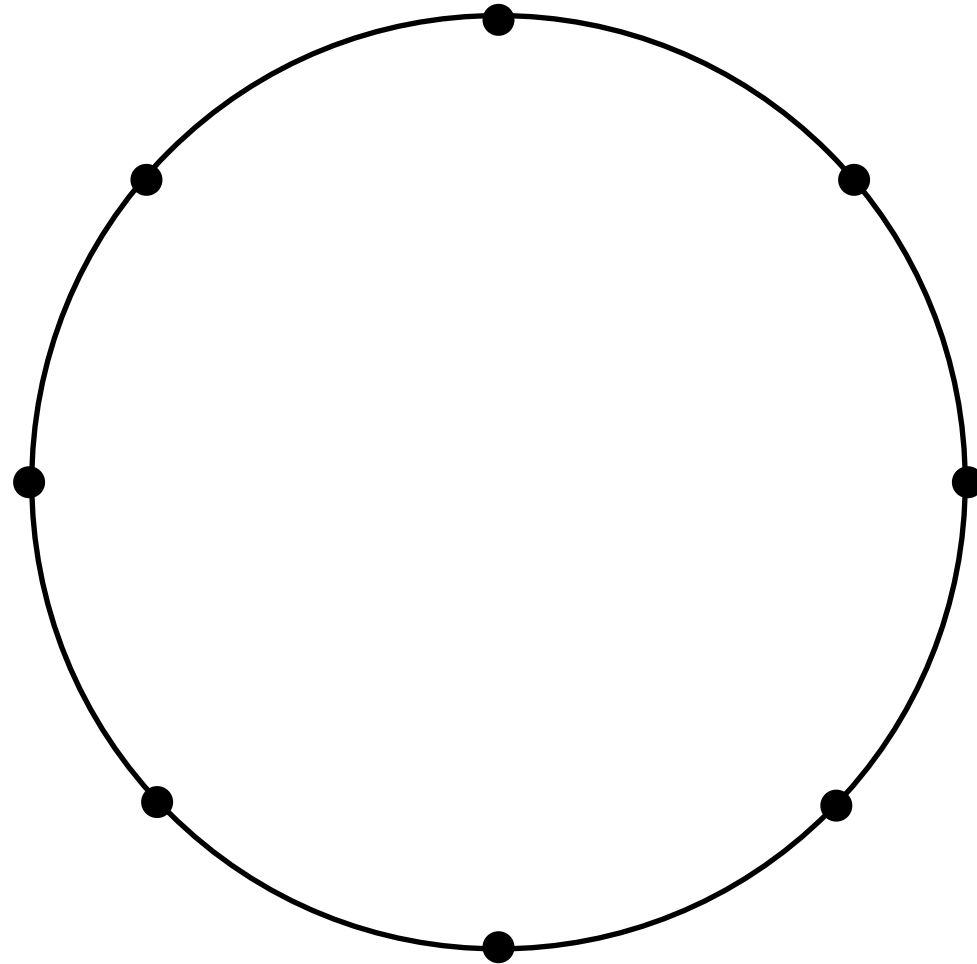
Chord

- ▶ The consistent hash function assigns each node and key (represents an object) an m-bit identifier, say using SHA-256. (m is usually very large. Why?)

$$h(x) = \text{SHA256}(x) \bmod 2^m$$

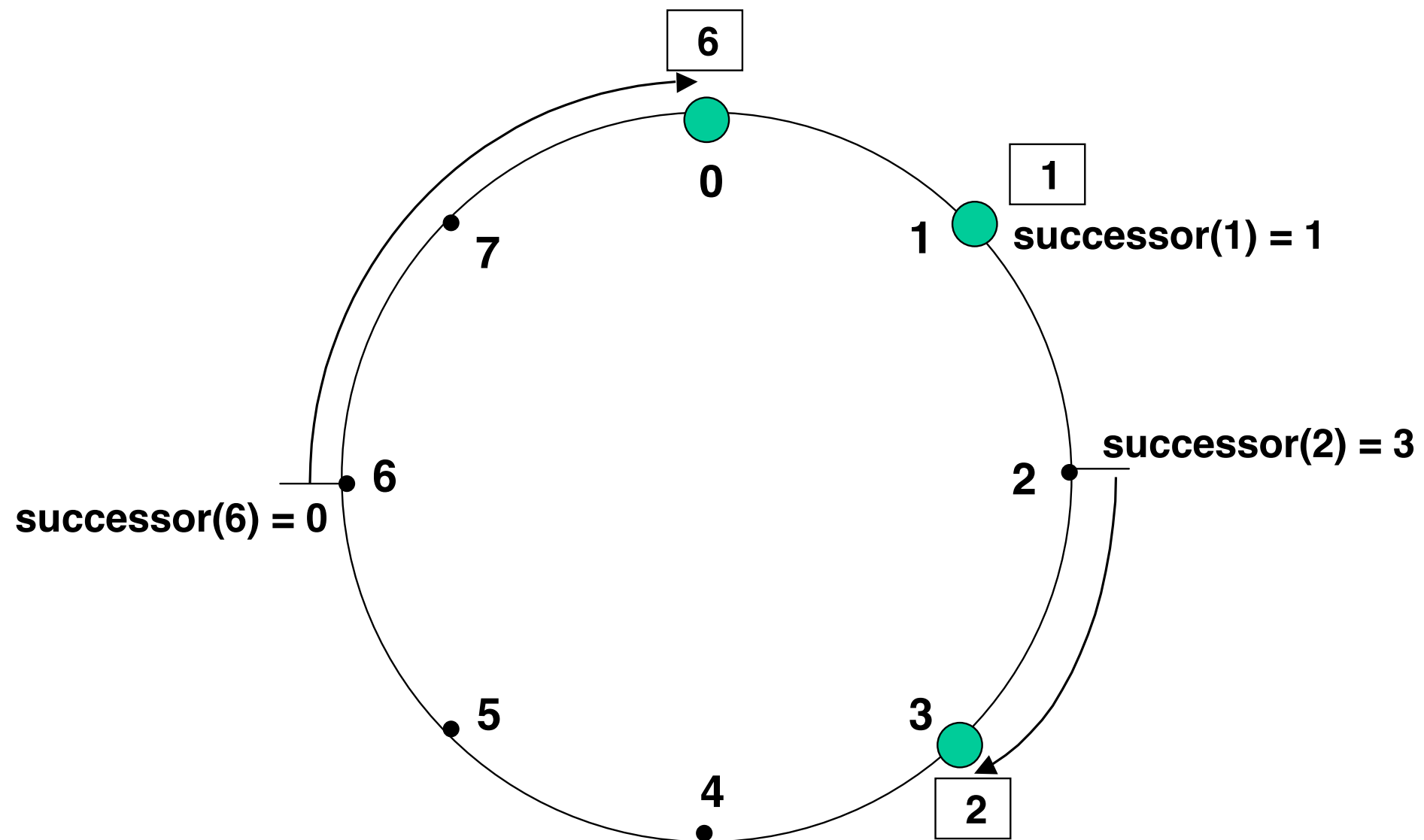
- ▶ Identifiers are ordered in an identifier circle modulo 2^m .

Chord



- ▶ Key k is assigned to the first node whose identifier is equal to or follows k in the circle. This node is called the successor node of key k , denoted by *successor(k)*.

Chord

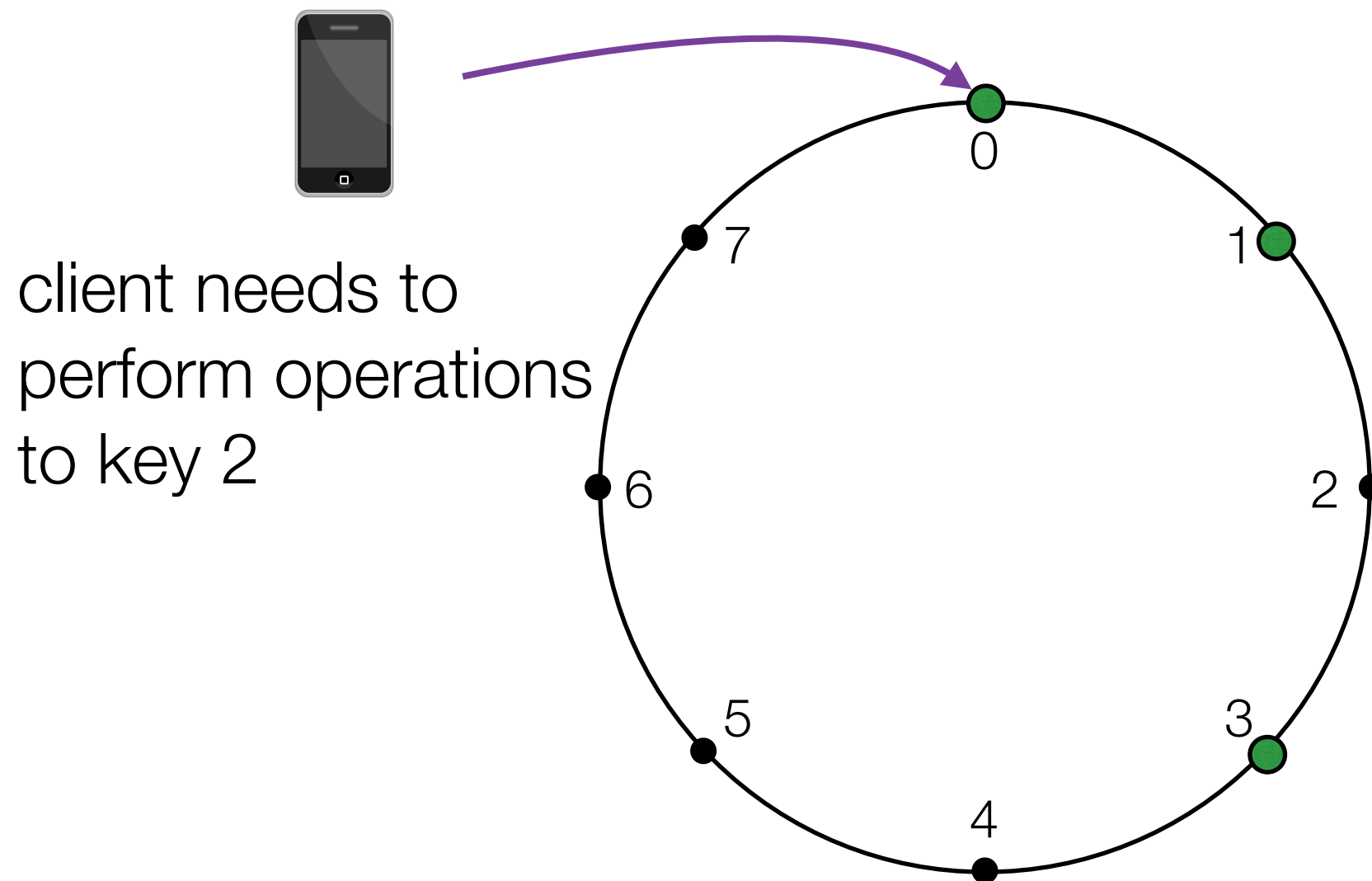


Chord – scalable key location

- ▶ A client connects to only one node, which probably isn't the successor of the object in question.
- ▶ There must be a distributed/scalable way for any node to figure out the successor of any key, to perform store/retrieve.

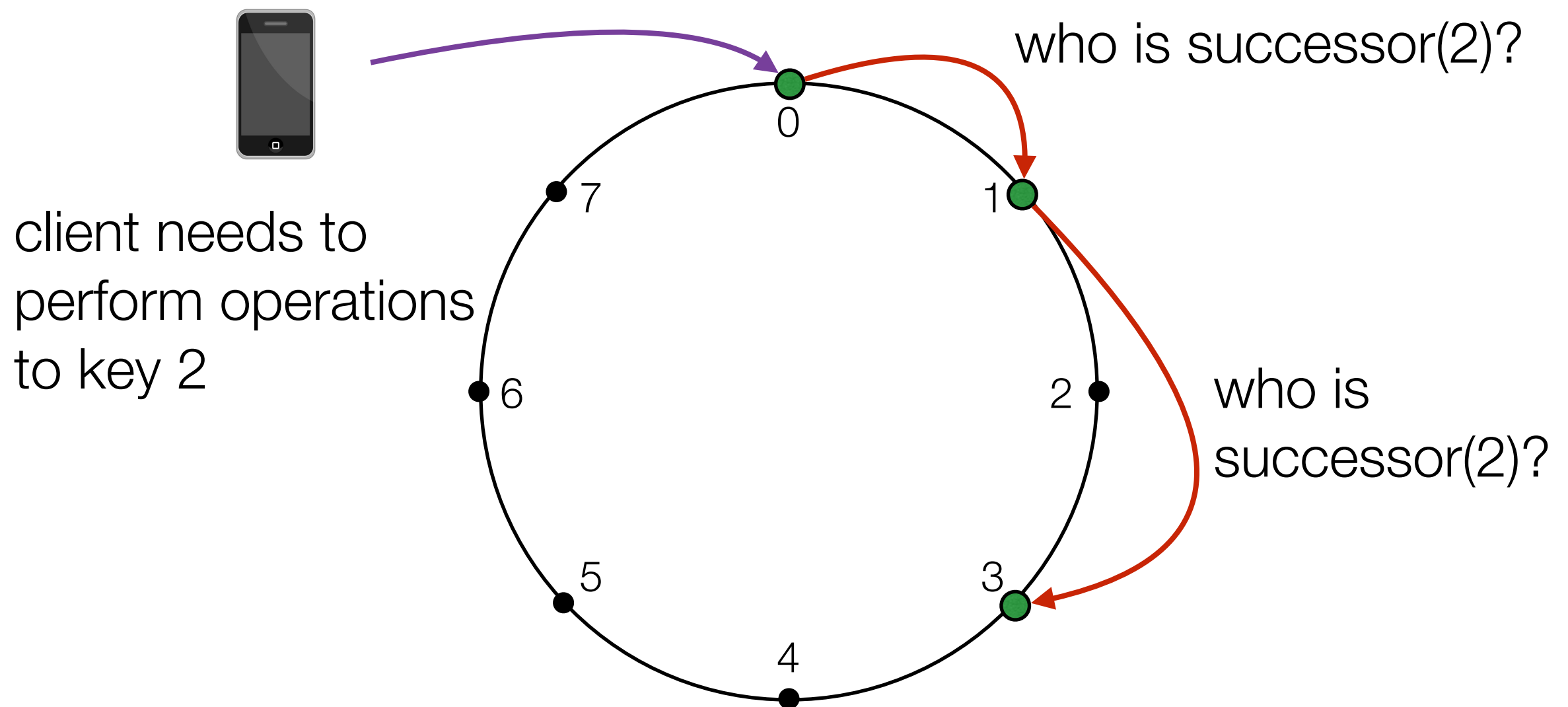
Chord – scalable key location

- ▶ Solution 1: each node only knows its successor node on the circle. Queries are passed around until they first encounter a node that succeeds the identifier.



Chord – scalable key location

- ▶ Solution 1: each node only knows its successor node on the circle. Queries are passed around until they first encounter a node that succeeds the identifier.



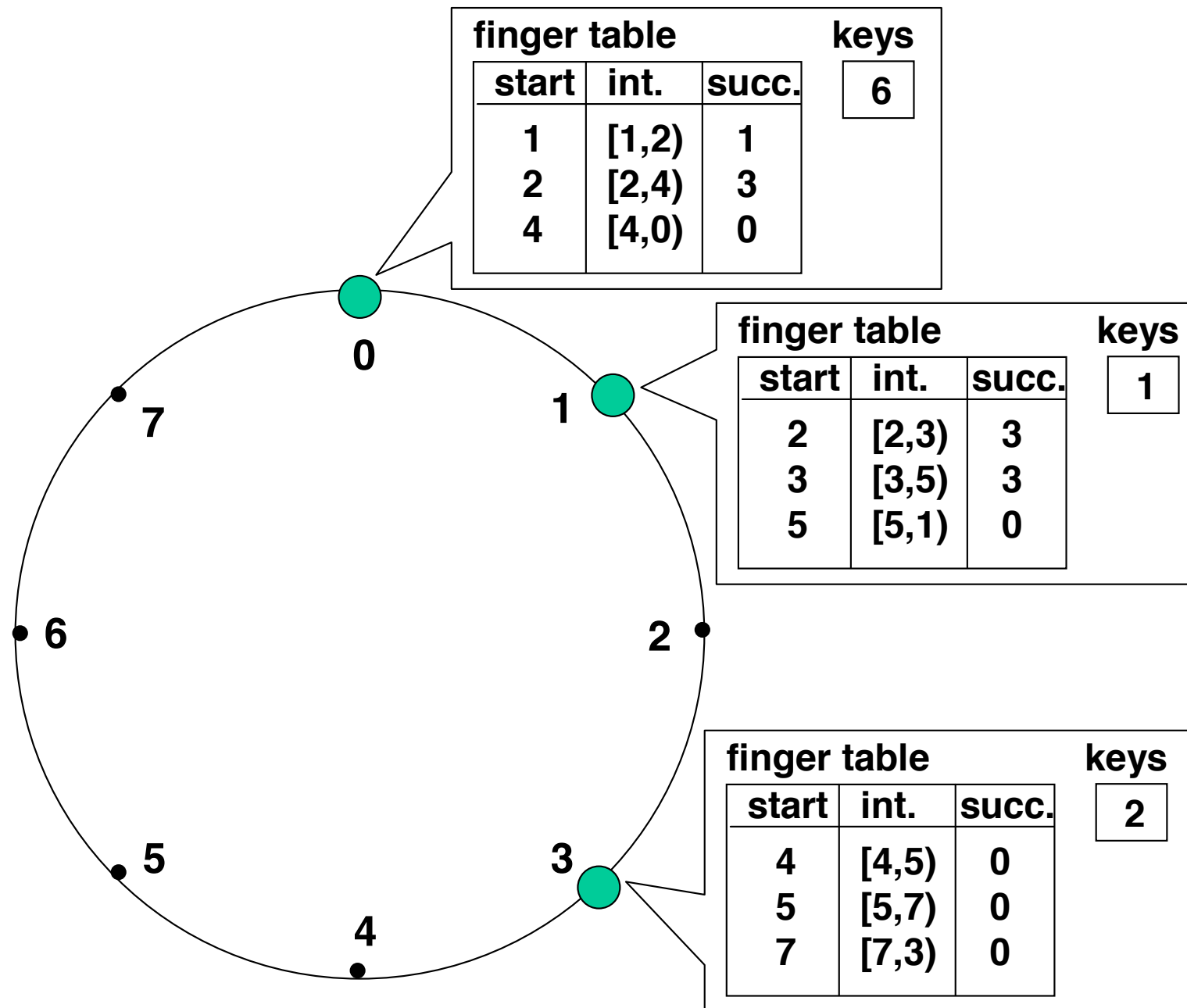
Chord – scalable key location

- ▶ Solution 1 is inefficient, as it may require traversing all N nodes to find the appropriate mapping.
- ▶ Solution 2: *finger tables*. Store additional info at each node to accelerate the location lookup.

Chord – finger table

- ▶ Each node, n , maintains a finger table with m entries.
- ▶ The i -th entry at node n contains the identity of $s = \text{successor}(n + 2^{i-1})$, its IP address and port number.
- ▶ Node s is called the i -th finger of node n .

Chord – finger table



Chord – key location lookup

- ▶ Each node only knows about m other nodes, and knows more about nodes closely following it than those farther away
- ▶ Each node doesn't have enough info to determine the successor of an arbitrary key.
- ▶ What if a node doesn't know the successor of a key k ? If it can find a node whose ID is closer than its own to k , that node will know more about the region of k .
- ▶ Node n searches its finger table for the node j whose ID most immediately precedes k , and asks j for the node it knows whose ID is closest to k .

Chord – lookup algorithm

// ask node n to find id 's successor

n .find_successor(id)

$n' = \text{find_predecessor}(id);$

return $n'.\text{successor};$

// ask node n to find id 's predecessor

n .find_predecessor(id)

$n' = n;$

while ($id \notin (n', n'.\text{successor}]$)

$n' = n'.\text{closest_preceding_finger}(id);$

return $n';$

// return closest finger preceding id

n .closest_preceding_finger(id)

for $i = m$ **downto** 1

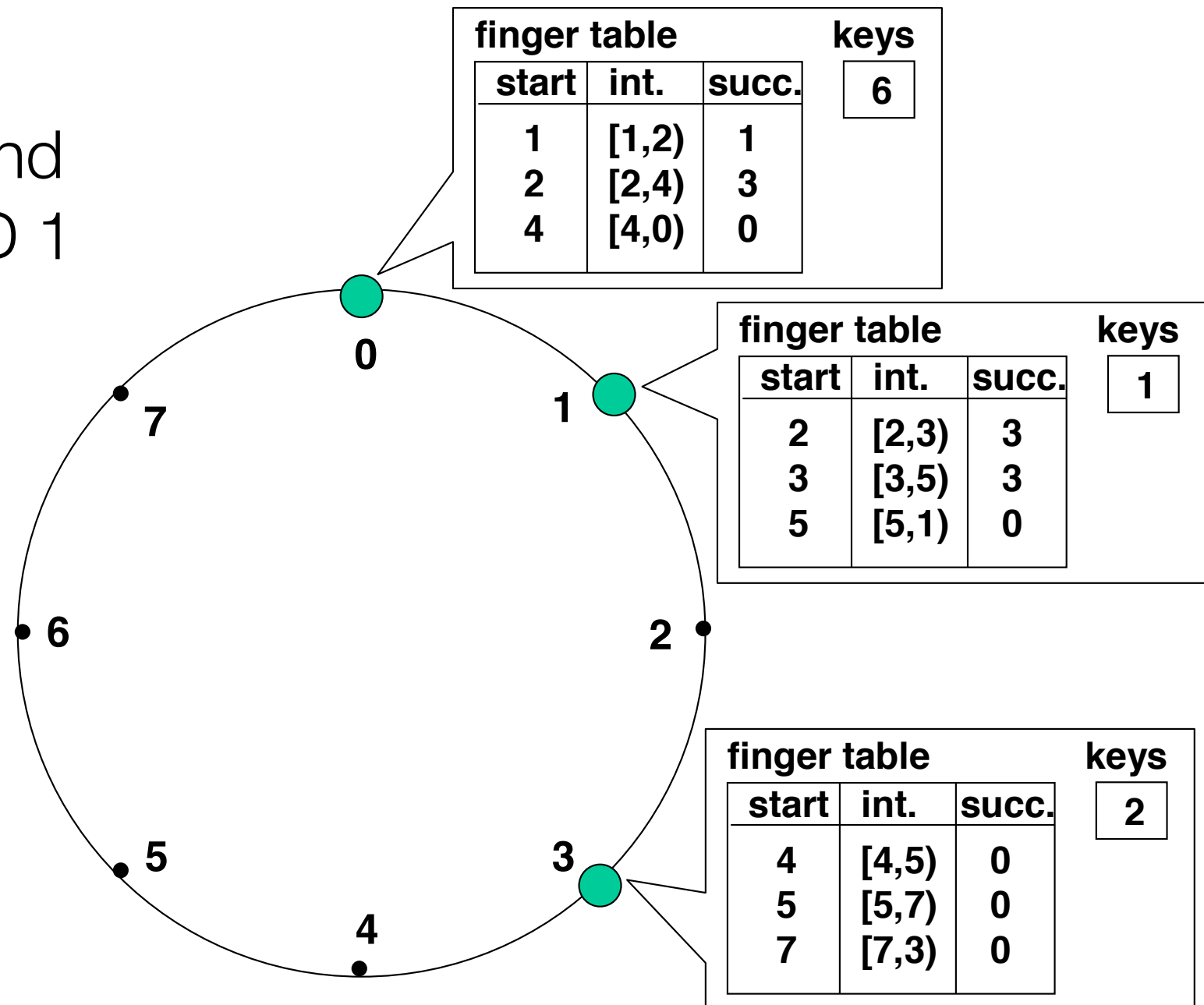
if ($\text{finger}[i].\text{node} \in (n, id)$)

return $\text{finger}[i].\text{node};$

return $n;$

Chord – lookup example

Node 3 wants to find the successor of ID 1



Node joins/departures can also be handled gracefully by Chord (re-distributing the keys, and reestablishing the finger tables)